

Grundlagen der Programmierung:

10. Generics und Container

Tutoren:

Daniel Grillmeyer, Ludwig Friedl, Jan Kerber, Jennifer Muck

1. Aufgaben zur Präzedenzregel in C
2. Wiederholung: Java Generics
3. Wiederholung: Java Container
4. Demo zu Containern
5. Hinweise zu Programmieraufgabe 4
6. Lösung: Programmieraufgabe 3

PRÄZEDENZREGEL

```
const unsigned int *volatile *(*next) (int a);
```

- A Bezeichner "name" (von links nach rechts) „[name] ist ein...“
- B Präzedenzreihenfolge:
- B.1 Klammer `()` „...“ gruppiert Teile einer Deklaration
- B.2 Postfixoperatoren:
- B.2.1 `()` „...Funktion mit Rückgabewert...“
- B.2.2 `[]` „...Array von...“
- B.3 Prefixoperator: `*` „...Pointer auf...“
- B.4 Prefixoperator `*` und `const` / `volatile` Modifizierer: „...[modifizierter] Pointer auf...“
- B.5 `const` / `volatile` Modifizierer neben Typ-Spezifikator: „...[modifizierter] [Spezifikator]“
- B.6 Typ-Spezifikator: „...[Spezifikator]“

```
const unsigned int *volatile *(*next) (int a);
```

- A Bezeichner "name" (von links nach rechts) „[name] ist ein...“
- B Präzedenzreihenfolge:
- B.1 Klammer `()` „...“ gruppiert
- B.2 Postfixoperatoren:
- B.2.1 `()` „...Funktion mit Rückgabewert...“
- B.2.2 `[]` „...Array von...“
- B.3 Prefixoperator: `*` „...Pointer auf...“
- B.4 Prefixoperator `*` und `const` / `volatile` Modifizierer: „...[modifizierter] Pointer auf...“
- B.5 `const` / `volatile` Modifizierer neben Typ-Spezifikator: „...[modifizierter] [Spezifikator]“
- B.6 Typ-Spezifikator: „...[Spezifikator]“

**Übergabeparameter
(falls vorhanden)
nicht vergessen!**

```
const unsigned int *volatile *(*next) (int a);
```

1.	A	next	„next ist ein...“
----	---	------	-------------------

```
const unsigned int *volatile *(*next) (int a);
```

1.	A	next	„next ist ein...“
2.	B.3	*	„...Pointer auf...“

```
const unsigned int *volatile *(*next) (int a);
```

1.	A	next	„next ist ein...“
2.	B.3	*	„...Pointer auf...“
3.	B.1	()	„...“


```
const unsigned int *volatile *(*next) (int a);
```

1.	A	next	„next ist ein...“
2.	B.3	*	„...Pointer auf...“
3.	B.1	()	„...“
4.	B.2.1	(int a)	„...eine Funktion mit Übergabeparameter Integer a und mit Rückgabewert ...“

```
const unsigned int *volatile *(*next) (int a);
```

1.	A	next	„next ist ein...“
2.	B.3	*	„...Pointer auf...“
3.	B.1	()	„...“
4.	B.2.1	(int a)	„...eine Funktion mit Übergabeparameter Integer a und mit Rückgabewert ...“
8.	B.3	*	„...Pointer auf...“

```
const unsigned int *volatile *(*next) (int a);
```

1.	A	next	„next ist ein...“
2.	B.3	*	„...Pointer auf...“
3.	B.1	()	„...“
4.	B.2.1	(int a)	„...eine Funktion mit Übergabeparameter Integer a und mit Rückgabewert ...“
8.	B.3	*	„...Pointer auf...“
9.	B.4	*volatile	„...einen volatile Pointer auf...“

```
const unsigned int *volatile *(*next) (int a);
```

1.	A	next	„next ist ein...“
2.	B.3	*	„...Pointer auf...“
3.	B.1	()	„...“
4.	B.2.1	(int a)	„...eine Funktion mit Übergabeparameter Integer a und mit Rückgabewert ...“
8.	B.3	*	„...Pointer auf...“
9.	B.4	*volatile	„...einen volatile Pointer auf...“
10.	B.6	const unsigned int	„...ein read-only unsigned int.“

```
2 float* (*add) (int x, const int* y);
```

1.

add

„add ist ein...“

```
2 float* (*add) (int x, const int* y);
```

1.	add	„add ist ein...“
2.	*	„...Pointer auf...“

```
2 float* (*add) (int x, const int* y);
```

1.	add	„add ist ein...“
2.	*	„...Pointer auf...“
3.	()	„...“

```
2 float* (*add) (int x, const int* y);
```

1.	add	„add ist ein...“
2.	*	„...Pointer auf...“
3.	()	„...“
4.	()	„...eine Funktion mit den Parametern...“


```
2 float* (*add) (int x, const int* y);
```

1.	add	„add ist ein...“
2.	*	„...Pointer auf...“
3.	()	„...“
4.	()	„...eine Funktion mit den Parametern...“
5.	x	„...x ist ein...“

```
2 float* (*add) (int x, const int* y);
```

1.	add	„add ist ein...“
2.	*	„...Pointer auf...“
3.	()	„...“
4.	()	„...eine Funktion mit den Parametern...“
5.	x	„...x ist ein...“
6.	int	„...int und...“

```
2 float* (*add) (int x, const int* y);
```

1.	add	„add ist ein...“
2.	*	„...Pointer auf...“
3.	()	„...“
4.	()	„...eine Funktion mit den Parametern...“
5.	x	„...x ist ein...“
6.	int	„...int und...“
7.	y	„...y ist ein...“

```
2 float* (*add) (int x, const int* y);
```

1.	add	„add ist ein...“
2.	*	„...Pointer auf...“
3.	()	„...“
4.	()	„...eine Funktion mit den Parametern...“
5.	x	„...x ist ein...“
6.	int	„...int und...“
7.	y	„...y ist ein...“
8.	*	„...Pointer auf...“

```
2 float* (*add) (int x, const int* y);
```

1.	add	„add ist ein...“
2.	*	„...Pointer auf...“
3.	()	„...“
4.	()	„...eine Funktion mit den Parametern...“
5.	x	„...x ist ein...“
6.	int	„...int und...“
7.	y	„...y ist ein...“
8.	*	„...Pointer auf...“
9.	const int	„...read-only int...“

```
2 float* (*add) (int x, const int* y);
```

1.	add	„add ist ein...“
2.	*	„...Pointer auf...“
3.	()	„...“
4.	()	„...eine Funktion mit den Parametern...“
5.	x	„...x ist ein...“
6.	int	„...int und...“
7.	y	„...y ist ein...“
8.	*	„...Pointer auf...“
9.	const int	„...read-only int...“
10.		„...und mit Rückgabewert...“

```
2 float* (*add) (int x, const int* y);
```

1.	add	„add ist ein...“
2.	*	„...Pointer auf...“
3.	()	„...“
4.	()	„...eine Funktion mit den Parametern...“
5.	x	„...x ist ein...“
6.	int	„...int und...“
7.	y	„...y ist ein...“
8.	*	„...Pointer auf...“
9.	const int	„...read-only int...“
10.		„...und mit Rückgabewert...“
11.	*	„...Pointer auf...“

```
2 float* (*add) (int x, const int* y);
```

1.	add	„add ist ein...“
2.	*	„...Pointer auf...“
3.	()	„...“
4.	()	„...eine Funktion mit den Parametern...“
5.	x	„...x ist ein...“
6.	int	„...int und...“
7.	y	„...y ist ein...“
8.	*	„...Pointer auf...“
9.	const int	„...read-only int...“
10.		„...und mit Rückgabewert...“
11.	*	„...Pointer auf...“
12.	float	„...ein float.“

JAVA GENERICS

- Generische Klassen enthalten Typ-Parameter

```
1 public class Pair<T> {  
2     private T first, second;  
3     public Pair(T f, T s) {  
4         first = f;  
5         second = s;  
6     }  
7     public T getFirst() {return first;}  
8     public T getSecond() {return second;}  
9 }
```

- Bei Verwendung muss dieser durch konkreten Typ ersetzt werden

```
10 Pair<Double> doublePair = new Pair<>(2.0, 3.0);
```

- Häufige Verwendung: Typisierte Container

```
11 List<Double> myList = new ArrayList<>(); // Liste kann nun Doubles enthalten
```

➤ Bestandteile generischer Klassen

```
1 public class Pair<T> {  
2     private T first, second;  
3     public Pair(T f, T s) {  
4         first = f;  
5         second = s;  
6     }  
7     public T getFirst() {return first;}  
8     public T getSecond() {return second;}  
9 }
```

Typparameter: dieser kann in der gesamten Klasse verwendet werden

Im Rest der Klasse wird an allen Stellen wo der Typ variabel ist der Typparameter benutzt, z.B. als

Typ einer Instanzvariable

Typ eines Methodenparameters

Rückgabebetyp einer Methode

- Bei Verwendung der Klasse mit einem Wert für T wird der Parameter entsprechend ersetzt und die Signaturen der Methoden werden konkret

```
1 public class Pair<T> {  
2     private T first, second;  
3     public Pair(T f, T s) {  
4         first = f;  
5         second = s;  
6     }  
7     public T getFirst() {return first;}  
8     public T getSecond() {return second;}  
9 }
```

T = Double

```
10 Pair<Double> doublePair = new Pair<>(2.0, 3.0);
```

- Bei Verwendung der Klasse mit einem Wert für T wird der Parameter entsprechend ersetzt und die Signaturen der Methoden werden konkret

```
1 public class Pair {  
2     private Double first, second;  
3     public Pair(Double f, Double s) {  
4         first = f;  
5         second = s;  
6     }  
7     public Double getFirst() {return first;}  
8     public Double getSecond() {return second;}  
9 }
```

T = Double

```
10 Pair<Double> doublePair = new Pair<>(2.0, 3.0);
```

(Code in Klasse Pair hier nur als Illustration verändert)

- Auch mehrere Typ-Parameter möglich

```
1 public class Pair<K, V> {  
2     private K first;  
3     private V second;  
4     public Pair(K f, V s) {  
5         first = f;  
6         second = s;  
7     }  
8     public K getFirst() {return first;}  
9     public V getSecond() {return second;}  
10 }
```

- Verwendung:

```
11 Pair<Integer, String> pair = new Pair<>(5, "Hello");
```

- Generische Methoden (auch in normalen Klassen) möglich

- Beispiel:

```
1 public class PairHelper {  
2     public static <T> boolean isEqual(Pair<T> p1, Pair<T> p2) {  
3         return p1.getFirst().equals(p2.getFirst())  
4             && p1.getSecond().equals(p2.getSecond());  
5     }  
6 }
```

- Verwendung:

```
7 Pair<Integer> p1 = new Pair<>(1, 2);  
8 Pair<Integer> p2 = new Pair<>(2, 2);  
9 PairHelper.<Integer>isEqual(p1, p2); //false
```

- Typ kann in der Regel weggelassen werden (Compiler erkennt benötigten Typ an Argumenten)

```
10 PairHelper.isEqual(p1, p2); //false
```

- Typ-Parameter können auch Einschränkungen unterliegen:

```

1 public class Vec2D<T extends Number> {
2     private T x, y;
3     public Vec2D(T x, T y) {
4         this.x = x;
5         this.y = y;
6     }
7     public double length() {
8         return Math.sqrt(x.doubleValue() * x.doubleValue()
9             + y.doubleValue() * y.doubleValue());
10    }
11 }

```

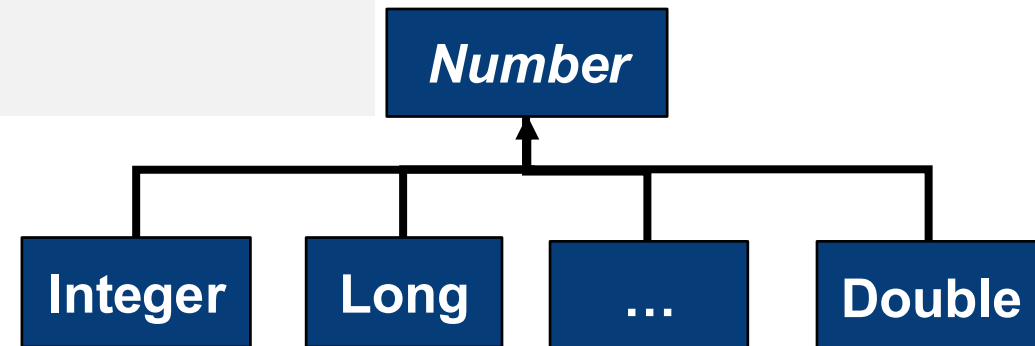
T muss Subklasse von Number sein
oder die Klasse Number

- Verwendung:

```

12 Vec2D<Integer> vec = new Vec2D<>(2, 3);
13 System.out.println(vec.length());

```



CONTAINER

1. Arrays

1. Eigenschaften

- Lineare Folge fester Größe
- length-Attribut
- Kann primitive Datentypen enthalten oder Referenzen auf Objekte

2. Vorteile

- Sehr effizient
- Typprüfung zur Compile-Zeit

3. Nachteile

- Feste Größe
- Nur ein Typ
- Wenige Methoden

2. Container

- Bieten komplexe Methoden zum Verwalten und Ändern von Objekten
- Zwei unterschiedliche Konzepte von Containern

1. Collection (Interface)

- Enthält einzelne Objekte
- Subinterfaces: z.B. List (geordnet) und Set (keine Duplikate)
- Konkrete Klassen: z.B. ArrayList, LinkedList, HashSet

2. Map (Interface)

- Enthält Schlüssel-Wert Paare
- Konkrete Klassen: z.B. HashMap, Hashtable

- Können verschiedene Typen beinhalten
- Keine feste Größe, automatische Größenanpassung
- Enthalten nur Referenzen auf Objekte
- Keine primitiven Typen (z.B. int – Verwendung von Hüllklasse Integer notwendig)
- Package `java.util`
- Beispiele von Methoden
 - `add(Object o)`
 - `remove(int index)`
 - `size()`
 - `get(int index)`
 - `put(Object key, Object value)`

➤ Interfaces

- Collection, List, Set und Map
→ z. B. Austausch von ArrayList zu LinkedList möglich, weil beide das Interface List implementieren

➤ Nachteile der Container

- Weniger effizient als Arrays
- Standardmäßig keine Typprüfung

➤ Bsp. für unterschiedliche Performance:

- Get: Array < ArrayList < LinkedList
- Iteration: Array < LinkedList < ArrayList
- Insert: Array < ArrayList < LinkedList
- Remove: Array < LinkedList < ArrayList

schneller < langsamer

➤ Vor Java 5.0:

```
1 List list = new LinkedList();  
2 list.add(5);  
3 int i = (int) list.get(0);
```

- Liste mit Object-Referenzen
- Kann Objekte beliebigen Typs enthalten
- Typecasts notwendig
- Mögliche `ClassCastException` zur Laufzeit

➤ Mit parametrisierten Typen (seit Java 5.0):

```
1 List <Integer> list  
    = new LinkedList<Integer>();  
2 list.add(5);  
3 int i = list.get(0);
```

- Liste mit Integer-Werten
- Typprüfung zur Übersetzungszeit
- Keine Typecasts notwendig

- Wildcards `?` bei der Verwendung von parametrisierten Klassen/Interfaces (hier: `List`) möglich:

```
1 public static double getFirst(List<? extends Number> list) {  
2     return list.get(0).doubleValue();  
3 }
```

Typ der Listenelemente muss
Subklasse von `Number` sein
oder die Klasse `Number`

- Bei Wildcards kann auch der `super`-Operator verwendet werden

```
4 public static void putDouble(List<? super Double> list, double value) {  
5     list.add(value);  
6 }
```

Typ der Listenelemente muss
Superklasse von `Double` sein
oder die Klasse `Double`

- Verwendung:

```
7 List<Number> numberList = new ArrayList<>();  
8 putDouble(numberList, 2.0);  
9 System.out.println(getFirst(numberList));
```

siehe  **IntelliJ IDEA**

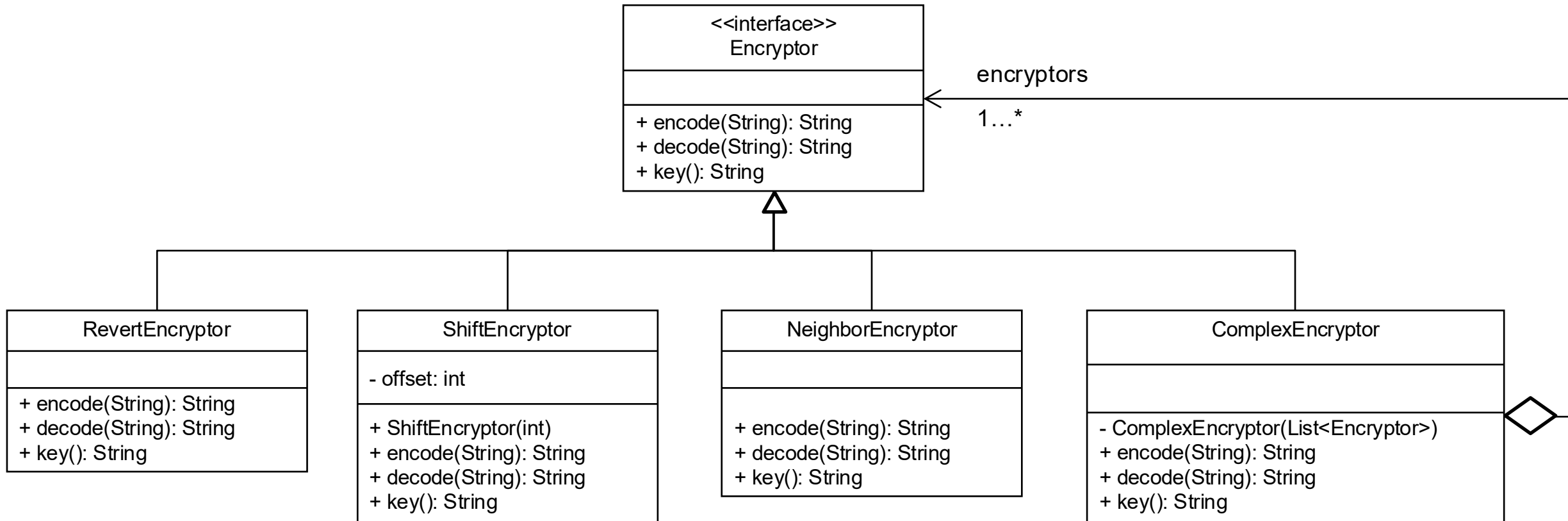
DEMO: CONTAINER

Crypto

PROGRAMMIERAUFGABE 4

Worum geht es?

➤ Implementierung von einfachen Verschlüsselungsverfahren



Revert Encryptor

- Text wird umgekehrt bzw. gespiegelt
- Gilt für Ver- und Entschlüsselung
- Beispiel: HELLO WORLD | DLROW OLLEH
- Als Key wird „RVT“ zurückgegeben

Shift Encryptor (a.k.a. Cäsar-Chiffre)

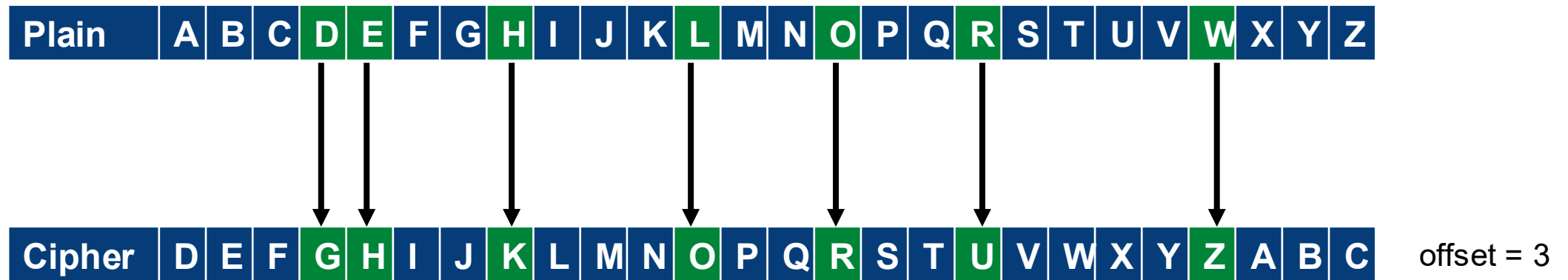
- Zu jedem Buchstaben im Text wird bei Verschlüsselung ein konstanter Offset addiert
- Bei der Entschlüsselung wird der Offset subtrahiert

Plain	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	
Cipher	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	offset = 1
Cipher	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	offset = 2
Cipher	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	offset = 3
...																											
Cipher	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	offset = 25

- Beispiel: „HELLO WORLD“ mit offset = 3

Shift Encryptor (a.k.a. Cäsar-Chiffre)

- Zu jedem Buchstaben im Text wird bei Verschlüsselung ein konstanter Offset addiert
- Bei der Entschlüsselung wird der Offset subtrahiert



- Beispiel: „HELLO WORLD“ mit offset = 3
→ „KHOOR ZRUOG“
- Als Key wird „SFT“ + offset zurückgegeben (z.B. „SFT3“ für offset = 3)

Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

- Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

- Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

- Verschlüsselung von „HAUS“ (Iteration über Text von links nach rechts)

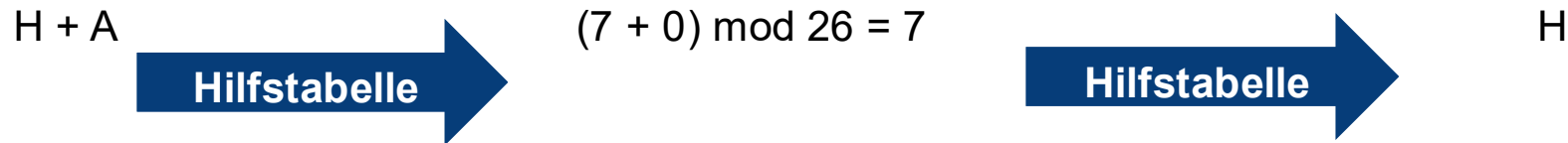
Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

- Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

- Verschlüsselung von „HAUS“ (Iteration über Text von links nach rechts)



Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

- Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

- Verschlüsselung von „HAUS“ (Iteration über Text von links nach rechts)

H + A

Hilfstabelle

$(7 + 0) \bmod 26 = 7$

Hilfstabelle

H

Damit der Zahlenwert
im Intervall 0, ..., 25
liegt

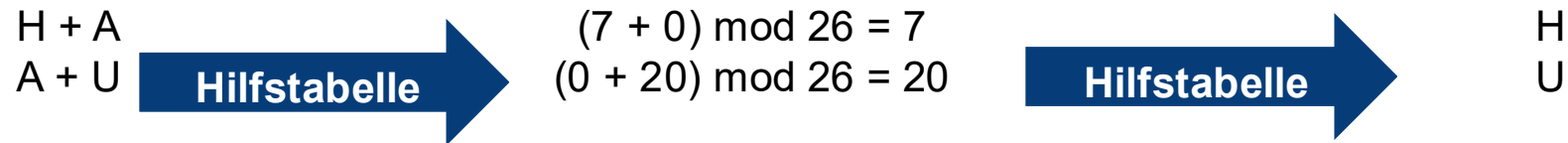
Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

- Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

- Verschlüsselung von „HAUS“ (Iteration über Text von links nach rechts)



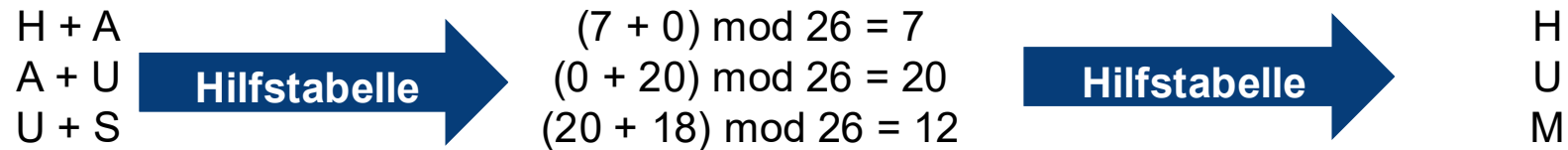
Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

- Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

- Verschlüsselung von „HAUS“ (Iteration über Text von links nach rechts)



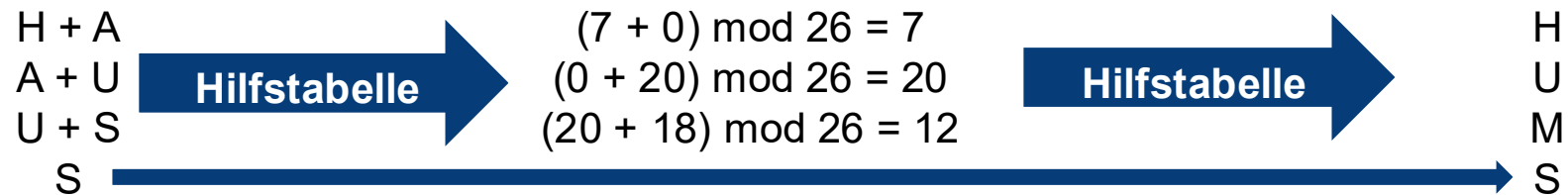
Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

- Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

- Verschlüsselung von „HAUS“ (Iteration über Text von links nach rechts)



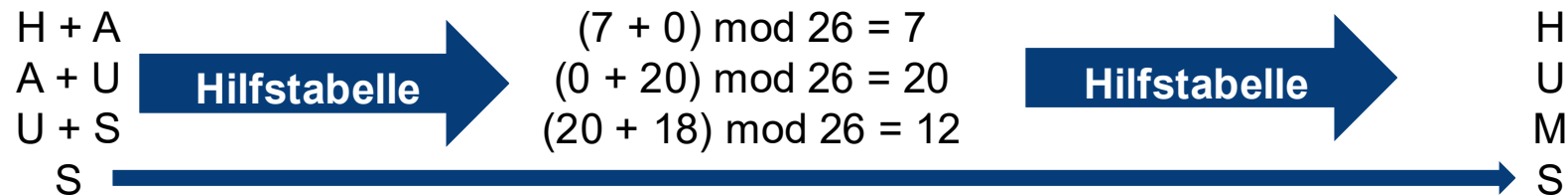
Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

- Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

- Verschlüsselung von „HAUS“ (Iteration über Text von links nach rechts)



- Entschlüsselung von „HUMS“ (Iteration über Text von rechts nach links, wir starten also mit „S“)

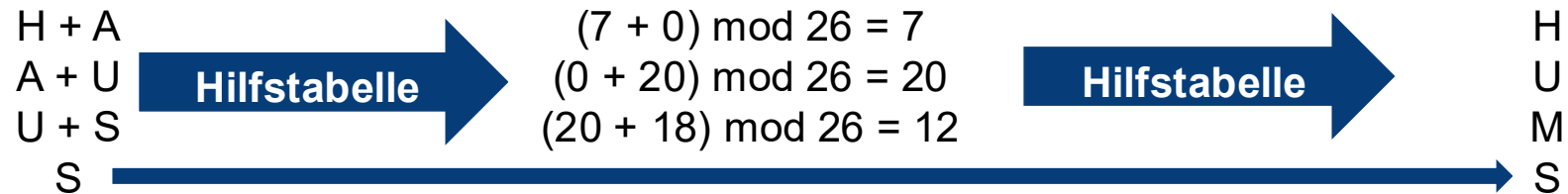
Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

- Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

- Verschlüsselung von „HAUS“ (Iteration über Text von links nach rechts)



- Entschlüsselung von „HUMS“ (Iteration über Text von rechts nach links, wir starten also mit „S“)



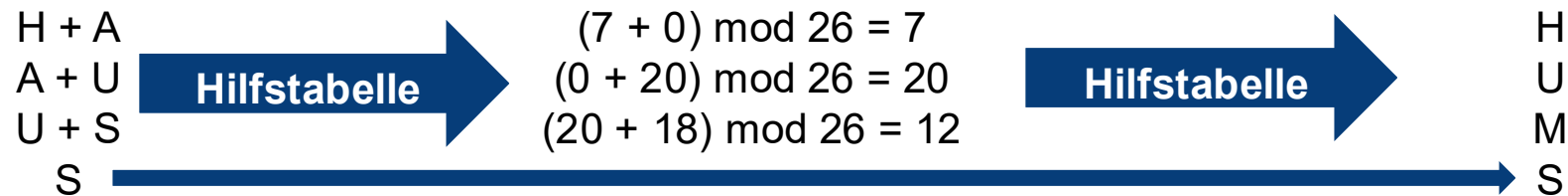
Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

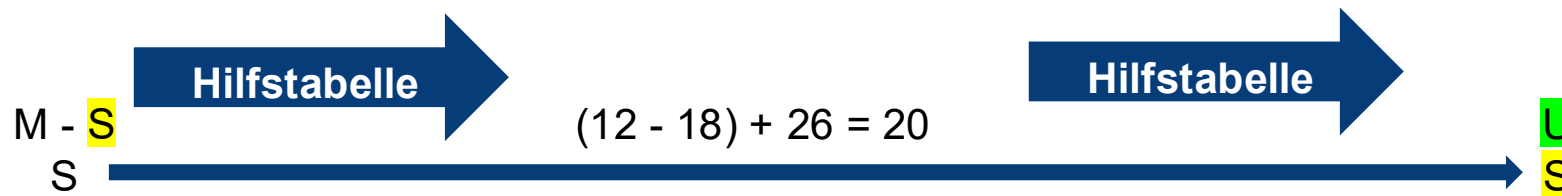
- Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

- Verschlüsselung von „HAUS“ (Iteration über Text von links nach rechts)



- Entschlüsselung von „HUMS“ (Iteration über Text von rechts nach links, wir starten also mit „S“)



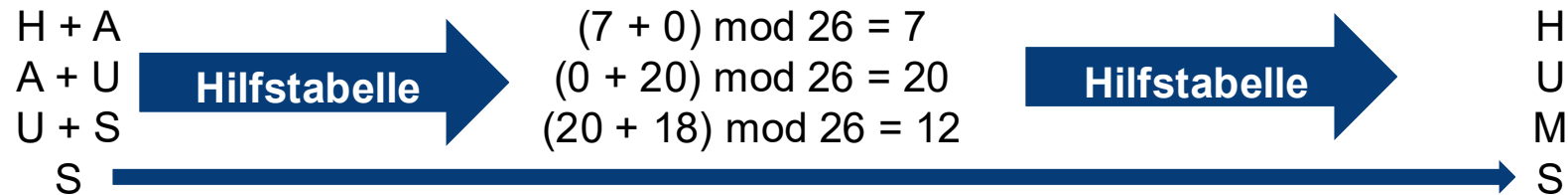
Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

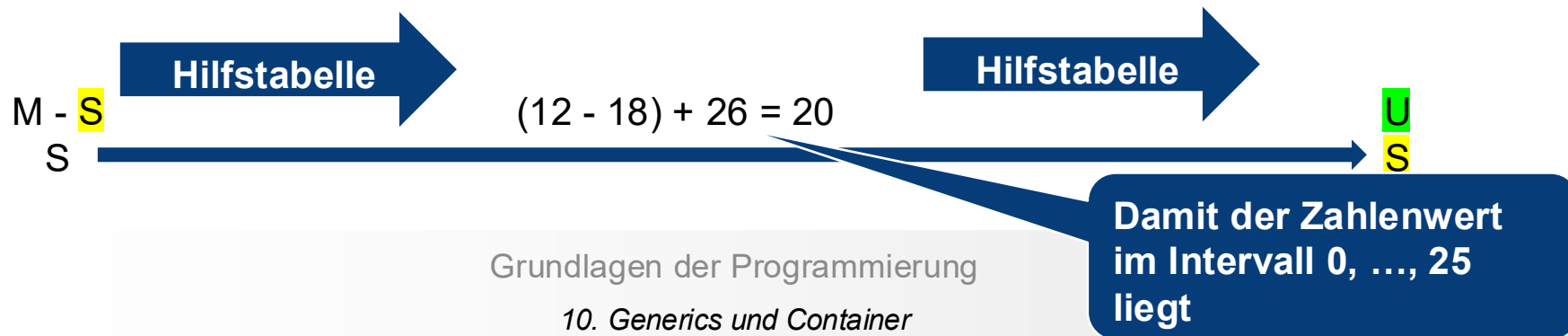
- Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

- Verschlüsselung von „HAUS“ (Iteration über Text von links nach rechts)



- Entschlüsselung von „HUMS“ (Iteration über Text von rechts nach links, wir starten also mit „S“)



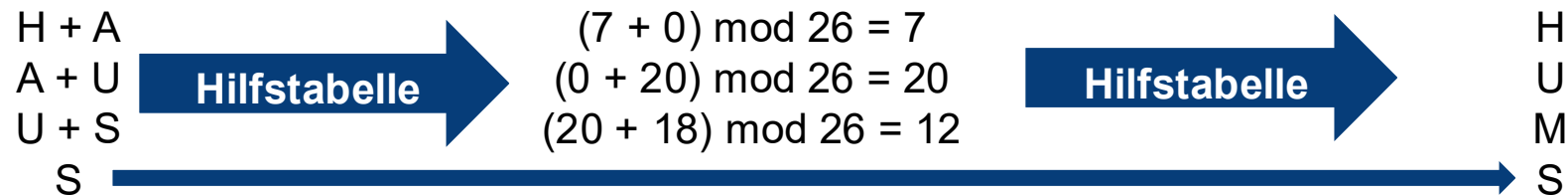
Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

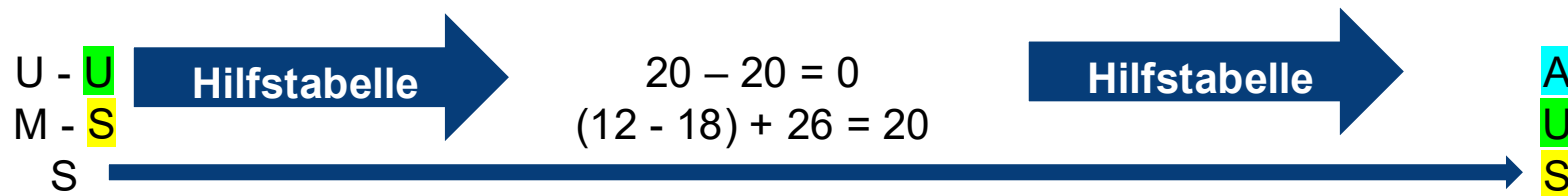
- Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

- Verschlüsselung von „HAUS“ (Iteration über Text von links nach rechts)



- Entschlüsselung von „HUMS“ (Iteration über Text von rechts nach links, wir starten also mit „S“)



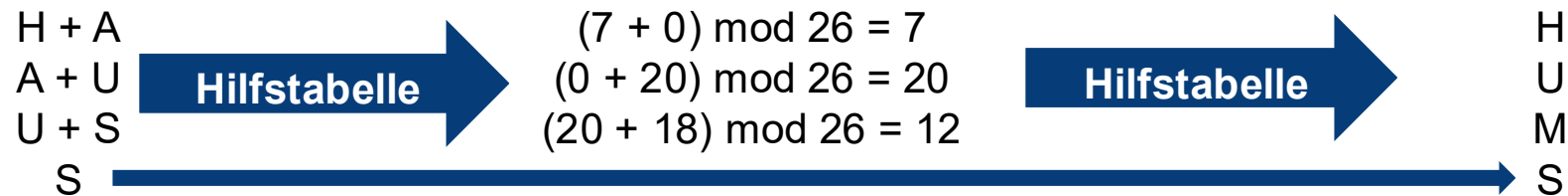
Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

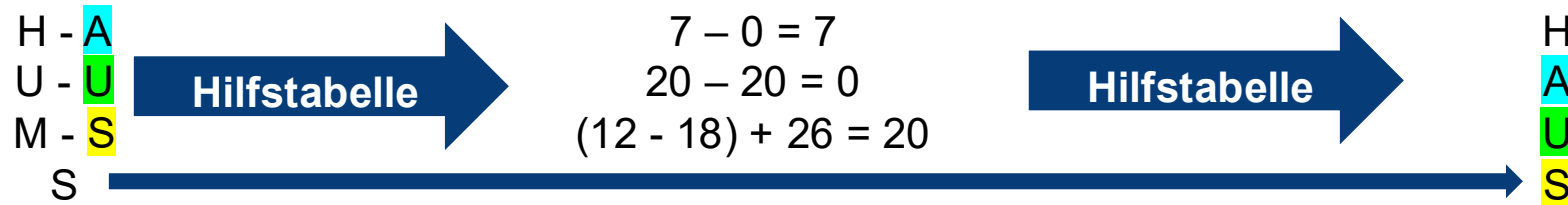
- Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

- Verschlüsselung von „HAUS“ (Iteration über Text von links nach rechts)



- Entschlüsselung von „HUMS“ (Iteration über Text von rechts nach links, wir starten also mit „S“)



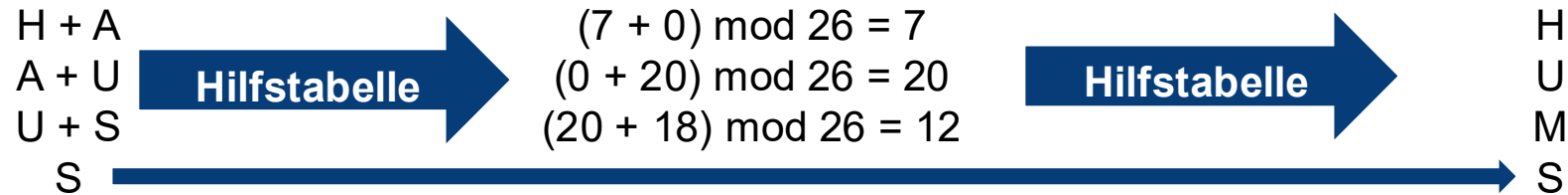
Neighbor Encryptor

- Jeder Buchstabe von A-Z wird auf einen Zahlenwert abgebildet

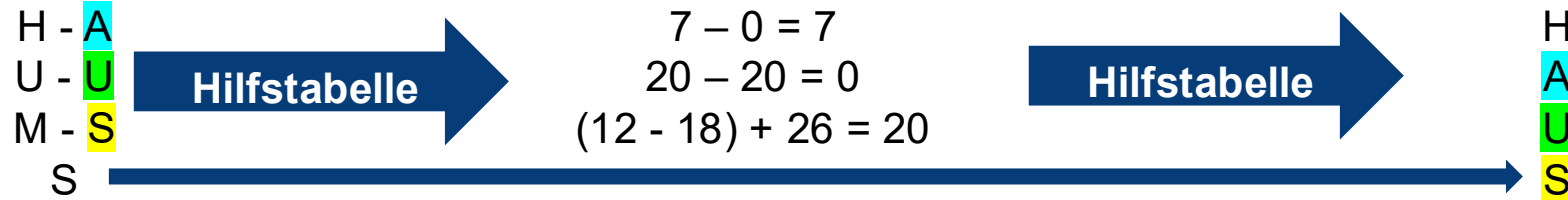
➤ Hilfstabelle:

Buchstabe	A	B	...	X	Y	Z
Zahlenwert	0	1		23	24	25

- Verschlüsselung von „HAUS“ (Iteration über Text von links nach rechts)



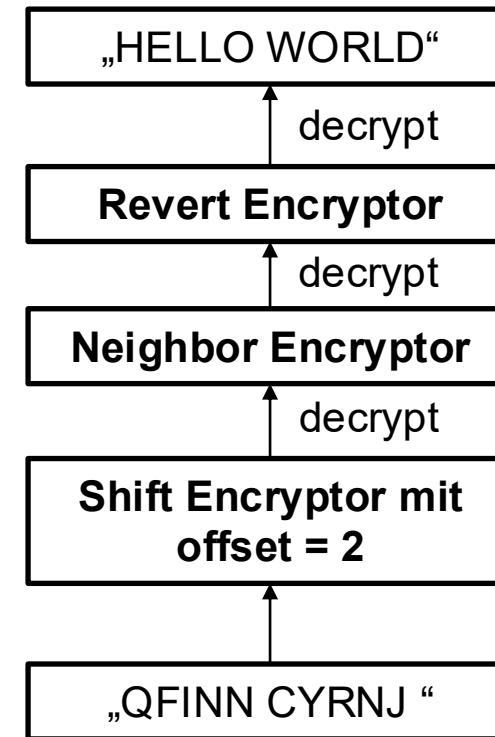
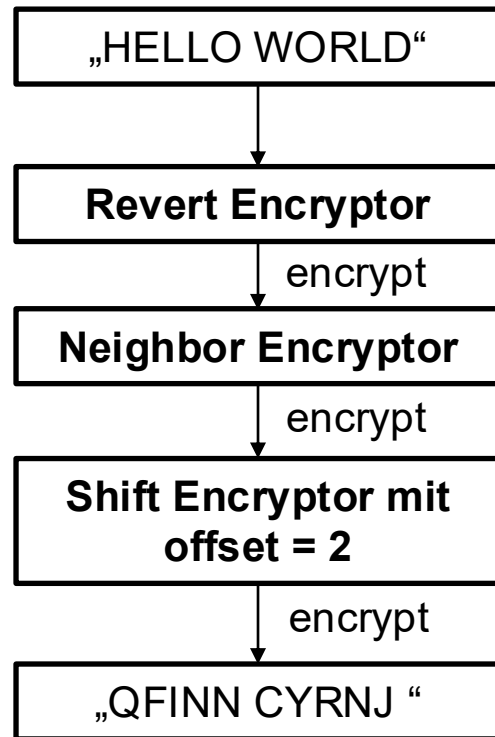
- Entschlüsselung von „HUMS“ (Iteration über Text von rechts nach links, wir starten also mit „S“)



- Als Key wird „NEI“ zurückgegeben

Complex Encryptor (1/2)

- Kombiniert die bisherigen Verfahren
- Schlüssel ergibt sich aus den Schlüsseln der einzelnen Verfahren
- Beispiel: Complex Encryptor mit dem Schlüssel „RVT-NEI-SFT2“:



Complex Encryptor (2/2)

```
1  /* import/package statements wurden hier weggelassen */
2  public class ComplexEncryptor implements Encryptor {
3      private final List<Encryptor> encryptors;
4      /* TODO: Implementierung von encode(), decode() und key()*/
5      private ComplexEncryptor(List<Encryptor> encryptors) { ... }
6
7
8
9
10
11
12
13
14
15
16 }
```

Zusammenbau einer Instanz von
ComplexEncryptor mit dem **Builder-Pattern**:

Complex Encryptor (2/2)

```
1  /* import/package statements wurden hier weggelassen */
2  public class ComplexEncryptor implements Encryptor {
3      private final List<Encryptor> encryptors;
4      /* TODO: Implementierung von encode(), decode() und key()*/
5      private ComplexEncryptor(List<Encryptor> encryptors) { ... }
6
7
8
9
10
11
12
13
14
15
16 }
```

Zusammenbau einer Instanz von ComplexEncryptor mit dem **Builder-Pattern**:

- Der Konstruktor ComplexEncryptor(...) ist **private** – keine direkte Instanziierung von außen erlaubt!

Complex Encryptor (2/2)

```
1  /* import/package statements wurden hier weggelassen */
2  public class ComplexEncryptor implements Encryptor {
3      private final List<Encryptor> encryptors;
4      /* TODO: Implementierung von encode(), decode() und key()*/
5      private ComplexEncryptor(List<Encryptor> encryptors) { ... }
6      public static Builder builder() { ... }
7
8
9
10
11
12
13
14
15
16 }
```

Zusammenbau einer Instanz von ComplexEncryptor mit dem **Builder-Pattern**:

- Der Konstruktor ComplexEncryptor(...) ist **private** – keine direkte Instanziierung von außen erlaubt!
- Methode public static builder(): Gibt Instanz von Builder zurück

Complex Encryptor (2/2)

```
1  /* import/package statements wurden hier weggelassen */
2  public class ComplexEncryptor implements Encryptor {
3      private final List<Encryptor> encryptors;
4      /* TODO: Implementierung von encode(), decode() und key()*/
5      private ComplexEncryptor(List<Encryptor> encryptors) { ... }
6      public static Builder builder() { ... }
7      public static class Builder {
8          private final List<Encryptor> encryptors;
9          private Builder() { ... }
10
11
12
13
14
15      }
16 }
```

Zusammenbau einer Instanz von ComplexEncryptor mit dem **Builder-Pattern**:

- Der Konstruktor ComplexEncryptor(...) ist **private** – keine direkte Instanziierung von außen erlaubt!
- Methode public static builder(): Gibt Instanz von Builder zurück
- Builder als geschachtelte Klasse: Hat Zugriff auf den Konstruktor ComplexEncryptor(...)

Complex Encryptor (2/2)

```
1  /* import/package statements wurden hier weggelassen */
2  public class ComplexEncryptor implements Encryptor {
3      private final List<Encryptor> encryptors;
4      /* TODO: Implementierung von encode(), decode() und key()*/
5      private ComplexEncryptor(List<Encryptor> encryptors) { ... }
6      public static Builder builder() { ... }
7      public static class Builder {
8          private final List<Encryptor> encryptors;
9          private Builder() { ... }
10         public Builder addEncryptor(Encryptor e) { ... }
11         public Builder addEncryptors(List<Encryptor> e) { ... }
12         public Builder removeEncryptor(Encryptor e) { ... }
13         public Builder reverse() { ... }
14     }
15 }
16 }
```

Zusammenbau einer Instanz von ComplexEncryptor mit dem **Builder-Pattern**:

- Der Konstruktor ComplexEncryptor(...) ist **private** – keine direkte Instanziierung von außen erlaubt!
- Methode public static builder(): Gibt Instanz von Builder zurück
- Builder als geschachtelte Klasse: Hat Zugriff auf den Konstruktor ComplexEncryptor(...)
- „Encryptors“ werden über die Instanzmethoden von Builder hinzugefügt/entfernt

Complex Encryptor (2/2)

```
1  /* import/package statements wurden hier weggelassen */
2  public class ComplexEncryptor implements Encryptor {
3      private final List<Encryptor> encryptors;
4      /* TODO: Implementierung von encode(), decode() und key()*/
5      private ComplexEncryptor(List<Encryptor> encryptors) { ... }
6      public static Builder builder() { ... }
7      public static class Builder {
8          private final List<Encryptor> encryptors;
9          private Builder() { ... }
10         public Builder addEncryptor(Encryptor e) { ... }
11         public Builder addEncryptors(List<Encryptor> e) { ... }
12         public Builder removeEncryptor(Encryptor e) { ... }
13         public Builder reverse() { ... }
14         public ComplexEncryptor build() { ... } ←
15     }
16 }
```

Zusammenbau einer Instanz von ComplexEncryptor mit dem **Builder-Pattern**:

- Der Konstruktor ComplexEncryptor(...) ist **private** – keine direkte Instanziierung von außen erlaubt!
- Methode public static builder(): Gibt Instanz von Builder zurück
- Builder als geschachtelte Klasse: Hat Zugriff auf den Konstruktor ComplexEncryptor(...)
- „Encryptors“ werden über die Instanzmethoden von Builder hinzugefügt/entfernt
- Zum Schluss wird build() aufgerufen und die fertige Instanz von ComplexEncryptor zurückgegeben

Complex Encryptor (2/2)

```
1  /* import/package statements wurden hier weggelassen */
2  public class ComplexEncryptor implements Encryptor {
3      private final List<Encryptor> encryptors;
4      /* TODO: Implementierung von encode(), decode() und key()*/
5      private ComplexEncryptor(List<Encryptor> encryptors) { ... }
6      public static Builder builder() { ... }
7      public static class Builder {
8          private final List<Encryptor> encryptors;
9          private Builder() { ... }
10         public Builder addEncryptor(Encryptor e) { ... }
11         public Builder addEncryptors(List<Encryptor> e) { ... }
12         public Builder removeEncryptor(Encryptor e) { ... }
13         public Builder reverse() { ... }
14         public ComplexEncryptor build() { ... }
15     }
16 }
```

Zusammenbau einer Instanz von ComplexEncryptor mit dem **Builder-Pattern**:

- Der Konstruktor ComplexEncryptor(...) ist **private** – keine direkte Instanziierung von außen erlaubt!
 - Methode public static builder(): Gibt Instanz von Builder zurück
 - Builder als geschachtelte Klasse: Hat Zugriff auf den Konstruktor ComplexEncryptor(...)
 - „Encryptors“ werden über die Instanzmethoden von Builder hinzugefügt/entfernt
 - Zum Schluss wird build() aufgerufen und die fertige Instanz von ComplexEncryptor zurückgegeben
- Verwendungsbeispiel auf der nächsten Folie!

Complex Encryptor (2/2)

```
1  /* import/package statements wurden hier weggelassen */
2  public class ComplexEncryptor implements Encryptor {
3      private final List<Encryptor> encryptors;
4      /* TODO: Implementierung von encode(), decode() und key()*/
5      private ComplexEncryptor(List<Encryptor> encryptors) { ... }
6      public static Builder builder() { ... }
7      public static class Builder {
8          private final List<Encryptor> encryptors;
9          private Builder() { ... }
10         public Builder addEncryptor(Encryptor e) { ... }
11         public Builder addEncryptors(List<Encryptor> e) { ... }
12         public Builder removeEncryptor(Encryptor e) { ... }
13         public Builder reverse() { ... }
14         public ComplexEncryptor build() { ... }
15     }
16 }
```

ComplexEncryptor.java

Verwendungsbeispiel zum Builder:

```
1  public class Main {
2      public static void main(String[] args) {
3
4          Encryptor ce = ComplexEncryptor.builder()
5              .addEncryptor(new ShiftEncryptor(2))
6              .addEncryptor(new NeighborEncryptor())
7              .addEncryptor(new RevertEncryptor())
8              .reverse()
9              .build();
10
11         System.out.println(ce.key());
12         // RVT-NEI-SFT2
13         System.out.println(ce.encode("HELLO WORLD"));
14         // QFINN CYRNJ
15     }
16 }
```

Main.java

Weitere Hinweise

- Ein paar nützliche Methoden für Strings:

```
1 String s = "HELLO WORLD";
2 s.charAt(1);           // 'E'
3 s.startsWith("HE");    // true
4 s.length();            // 11
5 s.toCharArray();      // [H, E, L, L, O,  , W, O, R, L, D]
6 s.substring(1,5);      // „ELLO“
```

- StringBuilder-Klasse eignet sich, um Strings „zusammenzubauen“:

```
1 StringBuilder sb = new StringBuilder("DLROW");
2 String s = sb
3     .append("OLLEH")
4     .insert(5, ' ')
5     .reverse()
6     .toString(); // „HELLO WORLD“
```

- Die Character von 'A' bis 'Z' entsprechen den int-Werten von 65 bis 90 („Unicode“)
 - ('Z' - 'A') == 25 // true

siehe  **IntelliJ IDEA**

LÖSUNG PROGRAMMIERAUFGABE 3

DIFFERENTIALRECHNUNG

Fragen vorbereiten

- Die beiden nächsten (und letzten) Übungen dienen primär der Klausurvorbereitung
- Bereitet für die nächsten Wochen Fragen für die Klausur vor, die ihr gerne diskutieren möchtet